ที่มาและความสำคัญของปัญหา (Problem Statement)

ช่องโหว่และข้อผิดพลาดสำคัญในระบบจองห้องประชุมแบบดั้งเดิมที่ต้องได้รับการแก้ไข

1. Double-Booking & Race Condition

การตรวจสอบเวลาว่างด้วย SELECT แล้วค่อย INSERT ผ่านแอปพลิเคชัน มีช่องโหว่เสี้ยววินาที ทำให้ผู้ใช้ 2 คนกดจองเวลาเดียวกันพร้อมกันแล้วทะลุเข้าฐานข้อมูลทั้งคู่ (ชนกันจริง)

2. Broken Access Control (ตรวจเฉพาะหน้าบ้าน)

ระบบส่วนใหญ่ซ่อนเฉพาะปุ่มบนหน้าเว็บ หากผู้ใช้ยิง HTTP POST ตรงๆ ไปยัง Endpoint หลังบ้าน จะสามารถแอบอนุมัติคำขอ หรือดูข้อมูลของผู้อื่นได้ (ขาด Server-side Enforcement)

3. Fake File & Storage Vulnerability

ตรวจสอบไฟล์แนบเฉพาะนามสกุล (.pdf) ทำให้ผู้ไม่หวังดีเปลี่ยนชื่อไวรัสหรือ Script (.exe/.sh) เป็น .pdf แล้วอัปโหลดขึ้นเซิร์ฟเวอร์ และเปิดดูผ่าน URL ตรงๆ ได้

4. Hard-coded Roles & ขาด Audit Trail

หากต้องการแก้สิทธิ์ต้องแก้โค้ดและ Restart Server ส่งผลกระทบต่อทั้งองค์กร อีกทั้งไม่มีบันทึกหลักฐานการใช้งาน (Audit Logs) ทำให้ไม่สามารถตรวจสอบย้อนหลังได้เมื่อมีข้อพิพาท

ภาพรวมระบบและวัตถุประสงค์ (System Overview)

วิสัยทัศน์และการออกแบบ RoomReserve เพื่อแก้ปัญหาอย่างเป็นระบบและปลอดภัย

Enterprise Security

แยก Logical Route **/app**, **/admin**, **/api** ชัดเจน ตรวจสอบสิทธิ์ที่ Middleware ฝั่ง Server ทุกเส้นทาง ป้องกัน Session Fixation, Brute Force และ IDOR

Zero Double-Booking

ควบคุม Concurrency ในระดับ Database Engine ด้วย **PostgreSQL Exclusion Constraint (GIST)** ป้องกันการจองชนกันอย่างเด็ดขาด 100% แม้ยื่นพร้อมกันระดับ Millisecond

Dynamic ACL Matrix

ปรับเปลี่ยนสิทธิ์ 14 รายการผ่านหน้าจอผู้ดูแลระบบได้ทันที **โดยไม่ต้อง Restart Server** พร้อมระบบ Live Version Sync ไปยัง Client ทุกเครื่องแบบ Real-time

คุณค่าสำคัญต่อองค์กร (Core Value)

เปลี่ยนกระบวนการจองห้องประชุมแบบกระดาษหรือแชตไลน์ ให้กลายเป็นระบบดิจิทัลที่มีวงจรอนุมัติ (Workflow) ชัดเจน มีการตรวจรับส่งมอบห้องหน้างาน และบันทึกประวัติความปลอดภัยครบถ้วนตามมาตรฐานสากล

4 Roles & Workflow State Machine

การแบ่งแยกหน้าที่ตามหลัก Separation of Duties และวงจรสถานะการทำงาน

Requester

- ยื่นจองห้องประชุม
- แนบไฟล์วาระการประชุม
- ดูเฉพาะรายการตนเอง
- ขอยกเลิกคำขอของตนเอง

Approver

- ตรวจสอบคำขอที่รอดำเนินการ
- อนุมัติ / ปฏิเสธพร้อมเหตุผล
- **Auto-Reject** คิวที่จองเวลาชนกันอัตโนมัติ
- Safety Rollback อนุมัติ

Facility Officer

- บล็อกปิดซ่อมบำรุงล่วงหน้า
- ส่งมอบห้อง (Check-in)
- ตรวจรับห้องและปิดงาน (Check-out) พร้อมแนบรูปถ่าย

Admin

- คุม Dynamic ACL Matrix สด
- จัดการผู้ใช้ / ระงับบัญชี
- ตรวจสอบ Audit Log ส่วนกลาง
- ป้องกันล็อกตัวเองออกจากระบบ



ขอบเขตของระบบ (In-Scope vs Out-of-Scope)

การกำหนดขอบเขตฟังก์ชันการทำงานที่ชัดเจนเพื่อประสิทธิภาพและความปลอดภัยสูงสุด

ขอบเขตภายในระบบ (In-Scope)

- ระบบยืนยันตัวตน Bcrypt Cost 12 + Persistent PostgreSQL Session Store
- ระบบป้องกัน Brute Force Login Lockout (8 ครั้งใน 10 นาที)
- Dynamic RBAC 14 Permissions พร้อมตารางสิทธิ์ Dynamic Matrix UI
- Database Engine Exclusion Constraint ป้องกัน Double-Booking
- ระบบ Auto-Reject คิวที่เวลาชนกันอัตโนมัติเมื่อมีการอนุมัติรายการใดรายการหนึ่ง
- การตรวจ Magic Bytes ของไฟล์แนบจริงฝั่ง Server + Anti-IDOR Stream
- บันทึกประวัติความปลอดภัยและการทำงาน (Security Audit Logging)

ขอบเขตนอกระบบสำหรับอนาคต (Out-of-Scope)

- ระบบชำระเงินค่าเช่าห้อง (เนื่องจากเป็นระบบบริหารจัดการภายในองค์กร)
- การเชื่อมต่อระบบปฏิทินบุคคลภายนอก (เช่น Google Calendar / Office 365)
- การควบคุมสวิตช์ไฟหรือกลอนประตูดิจิทัลฮาร์ดแวร์จริง (IoT Hardware)
- การส่ง SMS แจ้งเตือน (เลือกใช้ In-App Notifications และ Audit Logs แทน)

Route Architecture & Logical Separation

การแยกแยะเลเยอร์การทำงานอย่างเป็นระเบียบตามหลักสถาปัตยกรรม MVC

Web Application UI

/app — หน้าจอหลักผู้ใช้ (EJS)
/app/rooms — หน้าจัดการข้อมูลห้อง
/app/files/:id — เปิดดูไฟล์แนบ
เรนเดอร์แบบ SSR คลีนและรวดเร็ว

Web Admin Console

/admin/acl — Dynamic ACL Matrix
/admin/users — จัดการบัญชีผู้ใช้
/admin/logs — ดู Audit Logs ทั้งหมด
ล็อกสิทธิ์เฉพาะผู้มีสิทธิ์บริหาร

RESTful JSON API

/api/acl-version — ตรวจสอบสิทธิ์สด
/api/session-status — ตรวจสถานะล็อกอิน
/api/admin/users — จัดการผู้ใช้ (Zero Leak)
/api/admin/roles — อัปเดตสิทธิ์ ACL

```
router.use('/app', requireAuth, appRoutes);
router.use('/admin', requireAuth, adminRoutes);
router.use('/api/admin', requireAuth, requirePermission('user.manage'), apiAdminRoutes);
```

Middleware Pipeline (Request Lifecycle)

ลำดับการตรวจสอบความปลอดภัยฝั่ง Server ในทุกคำขอก่อนถึง Controller



Zero Bypass Principle: ทุกคำขอต้องผ่านการตรวจสอบตามลำดับอย่างเข้มงวด ป้องกันการลักลอบเรียกใช้งานฟังก์ชันโดยไม่ได้รับอนุญาต

Authentication & Session Management

มาตรฐานความปลอดภัยระดับองค์กร: การป้องกัน Brute Force, Fixation และ Credential Leakage

Bcrypt Cost 12 & Zero Leak

- รหัสผ่านเข้ารหัสด้วย **Bcrypt Salt Rounds = 12**
- ป้องกัน Rainbow Table และพจนานุกรมถอดรหัส
- **Zero Credential Leakage:** ตัด field `password_hash` ออกจาก JSON Response และ Audit Log ทุกจุด

PostgreSQL Session Store

- บันทึกลงตาราง **user_sessions** ผ่าน connect-pg-simple
- เซสชันไม่หลุดเมื่อรีสตาร์ตเซิร์ฟเวอร์
- กำหนด Session Timeout 30 นาที พร้อม Rolling Refresh เมื่อมีการใช้งานต่อเนื่อง

Session Regeneration

- เมื่อยืนยันตัวตนผ่าน จะสั่ง `req.session.regenerate()` ทันที
- สร้าง Session ID ใหม่ ป้องกัน **Session Fixation Attack**

Brute Force Lockout Guard

- ตรวจจับการพยายามล็อกอินผิดพลาดเกิน **8 ครั้งใน 10 นาที**
- ระงับการเข้าใช้งานชั่วคราว 10 นาทีทันที พร้อมบันทึก Security Audit Log

Database Schema & Exclusion Constraint

การออกแบบ 8 ตารางฐานข้อมูล PostgreSQL ผสานกลไก Concurrency Guard ระดับ Engine

1. roles (role_id, role_name)
2. permissions (permission_id, code)
3. role_permissions (M:N ACL Matrix)

4. users (user_id, role_id, is_active)
5. rooms (room_id, capacity, is_active)
6. user_sessions (sid, sess, expire)

7. room_bookings (ช่วงเวลา, สถานะ)
8. booking_attachments (UUID Storage)
9. audit_logs (บันทึกเหตุการณ์ความปลอดภัย)

```
-- Exclusion Constraint ระดับ Database Engine ป้องกัน Double-Booking:
CONSTRAINT no_overlapping_active_bookings EXCLUDE USING gist (
  room_id WITH =,
  tsrange(start_time, end_time) WITH &&
) WHERE (status IN ('APPROVED', 'IN_USE', 'BLOCKED'));
```

Dynamic ACL & RBAC Architecture

สถาปัตยกรรมการจัดการสิทธิ์แบบยืดหยุ่น ไร้การฮาร์ดโค้ดในระดับ Controller

Granular Permissions (14 สิทธิ์)

- **หมวดการจอง:** booking.create, booking.read_own, booking.cancel_own
- **หมวดอนุมัติ:** booking.approve
- **หมวดหน้างาน:** facility.checkin, facility.checkout, room.block_schedule
- **หมวดห้องประชุม:** room.manage, room.view_schedule
- **หมวดบริหารระบบ:** acl.manage, user.manage, audit.view, file.view

Dynamic Check on Request

- สิทธิ์ถูกผูกผ่านตาราง **role_permissions** ใน Database
- ฟังก์ชัน `requirePermission(code)` ดึงสิทธิ์ล่าสุดของ User แบบ Real-time
- หากไม่มีสิทธิ์ จะดีดออกด้วย HTTP 403 Forbidden ทันที พร้อมบันทึก Audit Log สถานะ DENIED

Dynamic ACL Matrix UI (Zero Downtime)

ปรับเปลี่ยนหน้าที่ของแต่ละตำแหน่งได้ทันทีผ่านหน้าเว็บ โดยไม่ต้อง Restart Server

2D Matrix Grid

ตาราง 2 มิติแสดงความสัมพันธ์ระหว่าง 4 Roles กับ 14 Permissions จัดกลุ่มตามโมดูลอย่างเป็นระเบียบ

Instant AJAX Toggle

คลิก Checkbox เพื่อเปิดหรือปิดสิทธิ์ ระบบยิง AJAX อัปเดตตารางฐานข้อมูลทันทีในเสี้ยววินาที

Live Client Sync

มี Endpoint `/api/acl-version` ตรวจสอบเวอร์ชัน ทำให้หน้าจอผู้ใช้ปรับเปลี่ยนตามสิทธิ์ใหม่ได้แบบ Real-time

พิสูจน์ได้ในการสาธิตสด: ปลดสิทธิ์ `booking.create` ของ `Requester` ปุ่มจองห้องบนหน้าเว็บหายทันที ยิง `API` โดน `403`

ACL Safety Guards & Integrity Rules

กลไกป้องกันความผิดพลาดของผู้ดูแลระบบ (Fool-Proof Protection)

ห้ามลบบทบาทที่มีผู้ใช้งานอยู่

หาก Role นั้นยังมีบัญชีผู้ใช้สังกัดอยู่ ระบบจะปฏิเสธการลบทันที ป้องกันการเกิดบัญชีผู้ใช้ลอยเคว้ง (Orphan Users) ที่ไม่มีสิทธิ์ในระบบ

ห้ามริบสิทธิ์จาก Admin คนสุดท้าย

ระบบมี Guard ป้องกันการปลดสิทธิ์ `acl.manage` จากผู้ดูแลระบบคนสุดท้าย เพื่อป้องกันอุบัติเหตุการล็อกตัวเองออกจากระบบ (Lockout Prevention)

Database Foreign Key Constraints

ใช้ `ON DELETE CASCADE` สำหรับตาราง role_permissions เพื่อล้างสิทธิ์อัตโนมัติ และใช้ `ON DELETE RESTRICT` สำหรับตารางข้อมูลหลัก ป้องกันข้อมูลสูญหาย

Secure File Upload & Magic Bytes Pipeline

การป้องกันการปลอมแปลงนามสกุลไฟล์และการเข้าถึงไฟล์โดยมิชอบ (Anti-IDOR)

Magic Bytes Guard

อ่าน 4–8 ไบต์แรกของไฟล์บนดิสก์ เช่น PDF ต้องขึ้นด้วย %PDF (0x25 0x50 0x44 0x46) หากเป็นไฟล์ปลอมจะ **ลบไฟล์ทิ้งทันที**

Outside Public Root

เก็บไฟล์ไว้ที่ `uploads/` นอก Web Public Root และเปลี่ยนชื่อเป็น Random UUID ป้องกัน Path Traversal (../../)

Anti-IDOR Stream

การเปิดดูไฟล์ต้องผ่าน Route `/app/files/:id` พร้อมเช็กสิทธิ์ `file.view` ในระบบ ห้ามเข้าถึงไฟล์โดยตรงผ่าน URL

```
if (!isValidMagicBytes) {
  fs.unlinkSync(filePath); // ลบไฟล์อันตรายทิ้งทันที
  await logAudit({ action: 'FILE_REJECTED', status: 'DENIED', details: 'Magic bytes mismatch' });
  return res.status(400).send('ตรวจพบลายเซ็นไฟล์ไม่ถูกต้อง');
}
```

Comprehensive Audit Logging

การบันทึกประวัติความปลอดภัยและการทำงานที่โปร่งใส ตรวจสอบย้อนหลังได้ 100%

ข้อมูลที่บันทึกลงใน audit_logs

- วันที่และเวลาประทับตรา (created_at)
- ผู้กระทำ (user_id, username, role_name)
- การกระทำ (action เช่น LOGIN_SUCCESS, BOOKING_CREATE, ACL_UPDATE)
- แหล่งข้อมูลเป้าหมาย (resource, resource_id)
- สถานะผลลัพธ์ (SUCCESS, FAILED, DENIED)
- ข้อมูลเครือข่าย (ip_address, user_agent)

มาตรการความปลอดภัยของ Log

- **Append-Only:** ไม่มี API หรือหน้าจอสำหรับแก้ไขหรือลบ Log
- **Zero Credential Leakage:** ไม่มีการบันทึกรหัสผ่านหรือ hash ใดๆ
- **Fast Query Indexing:** สร้าง B-tree Index บน created_at, action, username เพื่อค้นหาย้อนหลังได้อย่างรวดเร็ว

14 Automated Test Cases (npm test)

ผลการทดสอบระบบแบบอัตโนมัติครอบคลุมทุกมิติความปลอดภัย — ผ่าน 100% (14/14 Tests)

Test 1: Guest ยิง /api/admin/* ต้องได้ 401 Unauthorized
Test 2: Login รหัสผิด ต้อง Redirect กลับพร้อมข้อความ Error
Test 3: Requester ยิง /api/admin/* ต้องได้ 403 Forbidden
Test 4: Requester เปิดหน้า /admin/acl ต้องได้ 403 Forbidden
Test 5: Admin เข้า /api/admin/users ได้ 200 + Zero Credential Leak
Test 6: Admin เปิดหน้า /admin/acl ได้ 200 + Render Dynamic Matrix
Test 7: ผู้ใช้ที่ถูกระงับ (is_active=false) ต้องถูกบล็อกการล็อกอิน

Test 8: Session Regeneration on Login (Session ID เปลี่ยนใหม่)
Test 9: Brute Force Lockout Guard (ระงับเมื่อล็อกอินผิดเกินกำหนด)
Test 10: Dynamic ACL Live Sync Endpoint (/api/acl-version)
Test 11: File Upload Magic Bytes ปฏิเสธไฟล์ปลอมแปลงทันที
Test 12: File Upload Magic Bytes ยอมรับไฟล์ PDF แท้ (%PDF)
Test 13: Exclusion Constraint ป้องกัน Double-Booking ชนกัน
Test 14: Auto Collision Resolution (ปฏิเสธคิวชนอัตโนมัติเมื่ออนุมัติ)

Challenges & Lessons Learned

ความท้าทายทางวิศวกรรมซอฟต์แวร์ที่แก้ไขสำเร็จ และแนวทางการพัฒนาต่อยอด

ความท้าทายที่แก้ไขสำเร็จ

- **Double-Booking:** จัดการด้วย GIST Exclusion Constraint และ Pessimistic Row Lock (SELECT FOR UPDATE)
- **Session Loss on Reload:** ใช้ connect-pg-simple บันทึกลง PostgreSQL ทำให้เซสชันคงอยู่ตลอดการพัฒนา
- **Fake File Injection:** ใช้ Buffer Inspection อ่าน Magic Bytes ลึกถึงระดับไบต์จริงของไฟล์

การพัฒนาต่อยอดในอนาคต

- เชื่อมต่อ LINE Notify / Email แจ้งเตือนสถานะการอนุมัติแบบเรียลไทม์
- ระบบสแกน QR Code หน้าห้องประชุมเพื่อ Check-in ทันทีผ่านสมาร์ตโฟน
- เชื่อมต่อระบบ IoT เปิด/ปิดแอร์และไฟอัตโนมัติตามช่วงเวลาการจอง

Live Demo Guide (9 ขั้นตอนสาธิตสด 30/30)

ขั้นตอนการสาธิตสดต่อหน้าอาจารย์เพื่อยืนยันความถูกต้องของระบบครบทุกมิติ

1. **รัน npm test:** แสดงผลผ่าน 14/14 การทดสอบอัตโนมัติต่อหน้าอาจารย์
2. **สาธิต Brute Force:** ใส่รหัสผิด 8 ครั้ง แสดงการดีดออกและระงับชั่วคราว 10 นาที
3. **สาธิต Requester:** ล็อกอิน somchai ยื่นจองห้องผ่าน Mini Calendar Strip พร้อมแนบ PDF วาระประชุม
4. **สาธิตการจองชน (Pending):** ล็อกอินอีกแท็บ ยื่นจองห้องเดียวกัน เวลาชนกัน (ระบบอนุญาตให้ยื่น Pending ได้)
5. **สาธิต Approver (Auto-Reject):** ล็อกอิน manager อนุมัติคิวที่ 1 คิวที่ 2 ถูกปฏิเสธอัตโนมัติทันที
6. **สาธิต Facility:** ล็อกอิน facility กด Check-in เริ่มใช้ห้อง กด Check-out ตรวจรับห้องพร้อมแนบรูปภาพ
7. **สาธิต Dynamic ACL:** ล็อกอิน admin ปลดสิทธิ์ `booking.create` ของ Requester สดๆ หน้าเว็บ
8. **สาธิต Zero Downtime Effect:** สลับกลับแท็บ Requester ปุ่มจองหายทันที ยิง API โดน 403 (ไม่ต้องรีสตาร์ตเซิร์ฟเวอร์)
9. **สาธิต Audit Log & Magic Bytes:** แสดงหน้า Audit Log และทดสอบอัปโหลดไฟล์ปลอมแปลง ระบบปฏิเสธ 400 ทันที

สรุปผลโครงงาน RoomReserve (SmartSpace)

ระบบทำงานได้สมบูรณ์แบบตามเกณฑ์ประเมิน 30 คะแนนของอาจารย์ทุกข้อ
ครอบคลุมทั้ง 4 Roles, Dynamic ACL, Database Concurrency, File Security, Audit Logs และ 14 Automated Tests

Live Production URL: พร้อมใช้งานบน Render
GitHub Repository: ซอร์สโค้ดพร้อมคู่มือติดตั้งฉบับสมบูรณ์
Automated Tests: `npm test` (14/14 ผ่าน 100%)

Thank You • Questions & Answers (Q&A)